

Reviewer Pack — Minimal Rank of Quandle Structure over XOR-AND Tree Width

Entry 54c1c0c0a66c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 17:14:38 UTC

Minimal Rank of Quandle Structure over XOR-AND Tree Width

Entry ID: 54c1c0c0a66c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 17:14:38 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: XOR-AND Trees

Statement:

[‘For a given XOR-AND tree T with n leaves, the minimal rank of its quandle structure, defined as the minimum number of generators for a quandle on the set of nodes in T , is bounded by a function of the tree width $tw(T)$. Specifically, there exists a constant c such that the minimal rank $r_quandle(T)$ satisfies $r_quandle(T) \leq c * tw(T)$.’, ‘For all instances of XOR-AND trees with $n \leq 40$ leaves and tree width $tw(T)$, if $r_quandle(T) > c * tw(T)$, then T is not minimally quandles.’, ‘The quandle structure on an XOR-AND tree T is defined by the operation of combining nodes based on their parent-child relationships, where the operation is an associative, commutative, and idempotent operation.’]

Rationale (proposer’s reasoning):

[‘Quandle theory provides an algebraic structure that can capture certain properties of discrete objects. By linking quandle structures to XOR-AND trees, we may uncover new invariants or algorithms that could have implications for understanding complexity classes.’, ‘The minimal rank of a quandle is an important measure of its complexity, and bounding it by tree width could provide insights into the structure of XOR-AND trees and their computational properties.’, ‘Quandles have been applied to various fields such as geometry and group theory, but their application to complexity theory is relatively new and underexplored.’]

Taxonomy category: QUANDLE_TREE_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8d86d50368a0d960

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given XOR-AND tree T with $n \leq 40$ leaves, if the computed minimal rank $r_quandle(T)$ is greater than $c * tw(T)$, where c is a constant and $tw(T)$ is the tree width of T .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quandle theory" AND "XOR-AND trees" AND minimal rank" - "quandle structure" related TO "tree width" IN "XOR-AND trees" - "associative commutative idempotent operation" IN quandles ON XOR-AND trees

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0406024v1>] Layout of Graphs with Bounded Tree-Width - [<http://arxiv.org/abs/1506.07810v1>] Canonizing Graphs of Bounded Tree Width in Logspace - [<http://arxiv.org/abs/1211.7353v3>] Connected tree-width - [<http://arxiv.org/abs/2501.06791>] Classification of simple quandles of small order - [<http://arxiv.org/abs/math/0508351v2>] Symbolic computation with finite quandles - [<http://arxiv.org/abs/1505.03077v5>] Central extensions of groups and adjoint groups of quandles

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_xor_and_tree(n):
    if n == 1:
        return [0]
    left = generate_xor_and_tree(n // 2)
    right = generate_xor_and_tree(n - n // 2)
    return [left, right]

def tree_width(tree):
    if isinstance(tree, int):
        return 0
    left_width = tree_width(tree[0])
    right_width = tree_width(tree[1])
    return max(left_width, right_width) + 1

def quandle_rank(tree):
    if isinstance(tree, int):
        return 1
    left_rank = quandle_rank(tree[0])
    right_rank = quandle_rank(tree[1])
```

```

    return left_rank + right_rank - 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    results = []
    for n in range(5, 41):
        tree = generate_xor_and_tree(n)
        tw = tree_width(tree)
        r_quandle = quandle_rank(tree)
        results.append((n, tw, r_quandle))

    if len(results) < 30:
        return {
            "metric_name": "quandle_rank",
            "metric_value": None,
            "instances_tested": len(results),
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    total_r_quandle = sum(r for _, _, r in results)
    mean_r_quandle = Fraction(total_r_quandle, len(results))
    max_tw = max(tw for _, tw, _ in results)
    c = Fraction(mean_r_quandle, max_tw)

    support_count = 0
    counterexample = ""
    for n, tw, r_quandle in results:
        if r_quandle > c * tw:
            support_count += 1
        else:
            counterexample = f"n={n}, tw={tw}, r_quandle={r_quandle}"

    conjecture_holds = support_count / len(results) >= 0.8

    return {
        "metric_name": "quandle_rank",
        "metric_value": mean_r_quandle,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2**i - 1 for i in range(5, 30)]
    else:
        seeds = [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r['metric_value'] for r in results if r['metric_value'] is not None) / len(re

```

```

support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conjecture_holds'])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_428f42e5.py", line 88, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_428f42e5.py", line 44, in run_trial
    tw = tree_width(tree)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_428f42e5.py", line 28, in tree_width
    left_width = tree_width(tree[0])
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_428f42e5.py", line 28, in tree_width
    left_width = tree_width(tree[0])
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_428f42e5.py", line 29, in tree_width
    right_width = tree_width(tree[1])
                        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different input or investigate the cause of the crash to ensure the test can complete successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16326
2	preregistration	ollama_remote	glm4:latest	0	0	5714
3	novelty	ollama_remote	glm4:latest	0	0	5060
4	novelty	ollama_remote	glm4:latest	0	0	6395
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40825
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7684
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10642
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9655
9	judge	ollama_remote	glm4:latest	0	0	8110

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110411 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/54c1c0c0a66c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/54c1c0c0a66c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/54c1c0c0a66c.tar.gz` (if generated)